॥ श्रीगणेशाय नमः ॥

Ensemble Models

Ensemble : Multiple models are combined together.

↳ group

$m_1, m_2, \ldots, m_k$ → base models

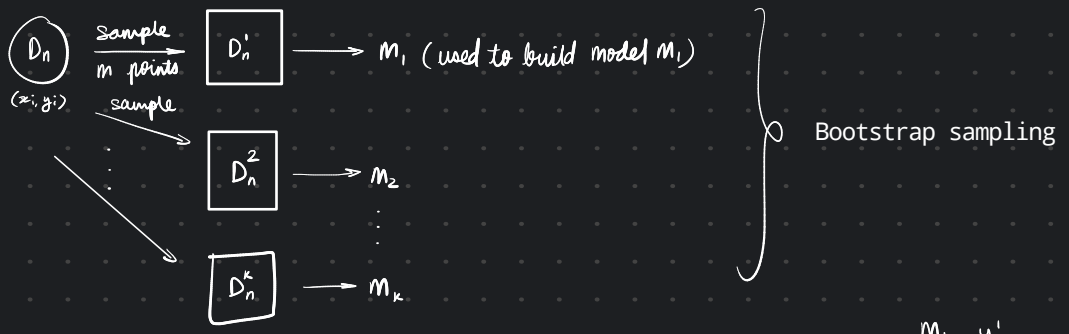
combine into a more powerful model M

The more different these base models are, the more better they can be combined.

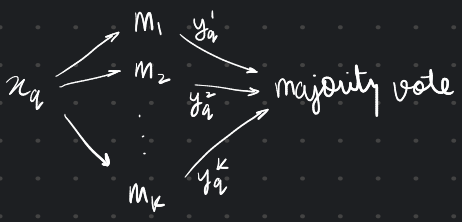
Types:
- Bagging (bootstrap aggregation)
- Boosting
- Stacking
- Cascading

Bagging (bootstrap aggregation)

Consider a dataset Dn (Xi, yi). We'll sample (with replacement) some points (say m points) from Dn. We train the model using those points. We'll do it k times.



We need to aggregate these models into a more powerful one.
For classification, a typical aggregation operation is majority vote.
For regression, we could compute the mean/median.



Here, every one of those models only sees a portion of the data.
Say the dataset has 10,000 points. A 100 of those are changed. The changed points will only come up in a subset of the sampled datasets. So the change in the original data would only affect a subset of the base models.
In a technique like majority vote, the aggregated result wouldn't be affected much.

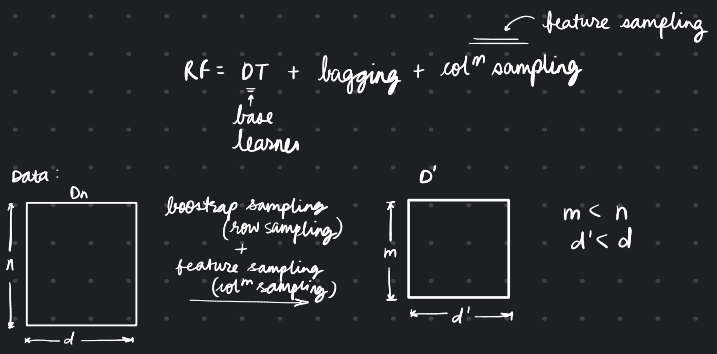
Bagging techniques can reduce the variance in a model without impacting the bias.

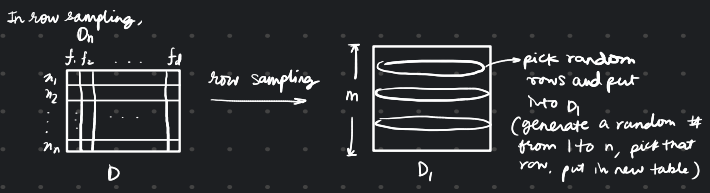
If the base models are low bias and high variance models, bagging them would give an aggregated model with a low bias as well as a reduced variance.

An example of a low-bias high-variance model is a decision tree of large depth.

Random forest

A random forest is bagging applied to a bunch of decision trees as base learners with column sampling.

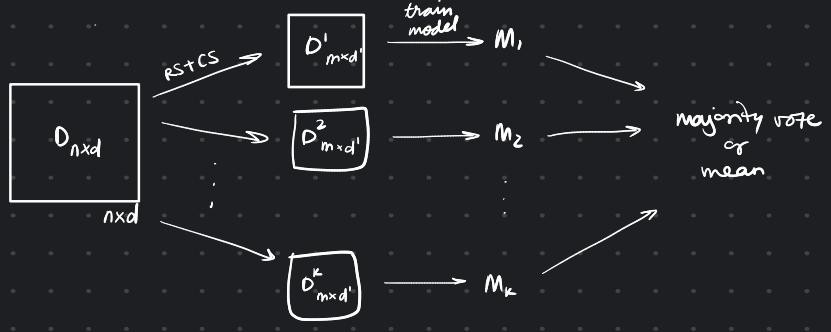




In column sampling,
only choose d' columns of those d columns in the new dataset.

So row + column sampling would give

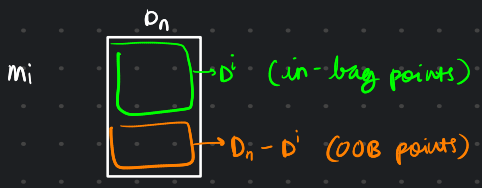




Each dataset is different in datapoints as well as features,
so models trained will be quite different. So very few models will get impacted by small changes in data.

In RFs, the base models M_i's are DTs of a reasonable depth.
- base models: should be low bias + high variance.
- reasonable depth: (often fully grown)

Out-of-bag points:
These are points not used by a model for learning.
Say model M_i uses points D^i.



These can be used for cross valida^n.
For model M^i,
OOB error is calculated on $D_n - D^i$.

Bias-variance tradeoff

Boosting

In bagging, we use low bias, high variance models + randomization (CS, RS) + aggregation.
↳ to reduce variance

In boosting, we use low variance, high bias models + additively combining.
↳ to reduce bias while keeping the variance low.

How does boosting reduce bias? The core idea:

$D_{train} = \{x_i, y_i\}_{j=1}^{n}$

stage 0: Build a model m_0 (high bias, low variance) on all of D_{train}.
↳ a shallow decision tree

Say m_0 fits a func^n $y = h_0(x)$. A high bias would mean a high train error.

subtract $h_0(x)$ from each y_i → non-trivial, since the bias is large.

$y_i - h_0(x_i) = error_i$

we have $\{x_i, y_i, error_i\}_{i=1}^{n}$

stage 1: Train m_1 with $\{x_i, error_i\}_{i=1}^{n}$. → $h_1(x_i) = error_i \approx y_i - h_0(x_i)$

$F_1(x) = \alpha_0 h_0(x) + \alpha_1 h_1(x)$
↳ model at the end of step 1
↳ weighted sum of 2 base models.

stage 2: $\{x_i, error_i\}$ → train m_2
↳ $y_i - F_1(x_i)$

$F_2(x) = \alpha_0 h_0(x) + \alpha_1 h_1(x) + \alpha_2 h_2(x)$

At the end of stage k,

$$F_k(x) = \sum_{i=0}^{k} \alpha_i h_i(x) \quad \left(\begin{matrix}\text{additive} \\ \text{weighted} \\ \text{model}\end{matrix}\right)$$

↳ each of the base models is trained to fit the residual error at the end of each stage $(i-1)$.

$h_i(x) \leftarrow \{x_i, err_i\}$ $\qquad y_i - f_{i-1}(x)$

The final model has a reduced training error.

Residuals, loss functions, and gradients

$$F_k(x) = \sum_{i=0}^{k} \alpha_i h_i(x)$$

Residual at the end of stage k $= err_i = y_i - F_k(x)$ $\qquad m_{k+1} \leftarrow \{x_i, err_i\}$

We define a loss function.

$$L(y_i, F_k(x_i)) = (y_i - F_k(x))^2$$

Say we're doing regression and are using a squared loss func^n.

$$\frac{\partial L}{\partial F_k(x_i)} = \frac{\partial L}{\partial z_i} = \frac{\partial (y_i - z_i)^2}{\partial z_i}$$

let $z_i = F_k(x_i)$

$$\frac{\partial L}{\partial z_i} = -2(y_i - z_i)$$

$$-\frac{\partial L}{\partial F_k(x_i)} = 2(y_i - F_k(x_i))$$

($y_i - F_k(x_i)$: residual)

negative gradient of loss func^n w.r.t. func^n at the end of stage k $\propto$ residual

So the neg-gradient of the loss is often termed as the pseudo-residual.

$\{x_i, err_i\}$ → $\{x_i, \text{pseudo-residual}\}$ → $-\frac{\partial L}{\partial (F_k(x))}$

we could use it to train at each stage.

Now we could minimize any differentiable loss function.

So at a stage i: we have $F_{i-1}(x)$

we train m_i $h_i(x)$ on $\{x_i, err_i\}$

pseudo-residual $\frac{\partial L}{\partial F_{i-1}(x)}$

Gradient boosting

Input: training set $\{(x_i, y_i)\}_{i=1}^{n}$, a differentiable loss function $L(y, F(x))$, number of iterations M.
↳ # of base learners.

Algorithm:

1. Initialize model with a constant value:

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{n} L(y_i, \gamma).$$

Find a γ that minimizes the loss func^n. say mean(y_i) for $L=(y_i-\gamma)^2$

2. For m = 1 to M:
   1. Compute so-called *pseudo-residuals*:

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \ldots, n.$$

   2. Fit a base learner (or weak learner, e.g. tree) closed under scaling $h_m(x)$ to pseudo-residuals, i.e. train it using the training set $\{(x_i, r_{im})\}_{i=1}^{n}$.
   3. Compute multiplier γ_m by solving the following one-dimensional optimization problem:

$$\gamma_m = \arg\min_{\gamma} \sum_{i=1}^{n} L\left(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)\right).$$

Find γ that minimizes $L + \gamma h_m(x_i)$ (single-variable optimizat^n)

   4. Update the model:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x).$$

3. Output $F_M(x)$.

$h_m(x)$, the base learners could be anything.
If they are decision trees, shallow since we want high bias, low variance (decision stumps), that's called a gradient boosted decision trees.

Regularization by shrinkage

$$F_M(x) = h_0(x) + \sum_{m=1}^{M} \gamma_m h_m(x)$$

M = # of base models.
↳ we'd find it using CV

As # of base models, chances of overfitting increase.

$\Rightarrow$ M ↑ variance ↑

Shrinkage:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

$$\downarrow$$

$$F_m(x) = F_{m-1}(x) + \nu \gamma_m h_m(x)$$

$0 \le \nu \le 1$ often $= 0.1$
↳ learning rate (hyperparameter)

So as ν ↓ variance ↓.
But computat° time ↑↑.

(ν, m) are used to control the bias-variance trade-off.
↓ shrinkage ↳ # of base models

Train and run-time complexity

For a decision tree, $T = O(n \lg d)$

⇒ Train time: $T = O(n \lg d * M)$ M = # of base learners

Unlike random forests, these models are not trivially parallelizable.

Run-time: $T = O(\underbrace{\text{depth}}_{\text{much small}} * M + M)$ ⇒ can be run in low latency applications.

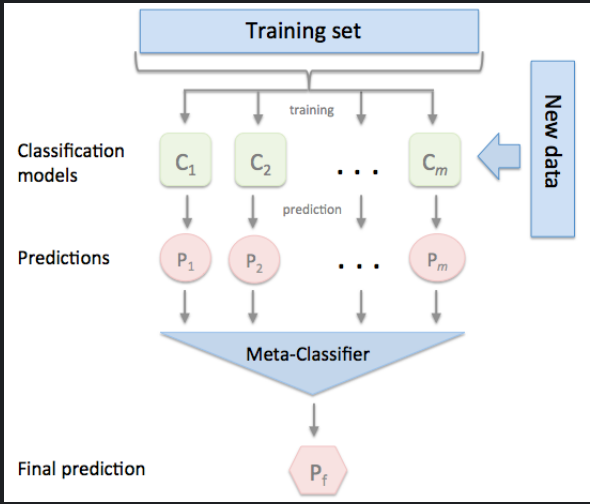
Space: Space to store each tree (stored as if-else conditions, which are highly optimized) + $\sqrt{m}$ for each model

Code

AdaBoost: adaptive boosting

It is often used in image-processing applications (face detection).

Stacking



We train n models from D_{train}. The different they are, the better. These are often trained with different algorithms.

A query point is passed to all the models $C_1, C_2, \dots C_m$ to get $y_{q_1}, y_{q_2}, \dots y_{q_m}$.

We build a model (a meta-classifier) on top of all these models. (a new model instead of aggregation).

The base models are well tuned models.

Algorithm 19.7 Stacking

Input: Training data $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^m$ ($\mathbf{x}_i \in \mathbb{R}^n$, $y_i \in \mathcal{Y}$)
Output: An ensemble classifier H

1: Step 1: Learn first-level classifiers
2: **for** $t \leftarrow 1$ to T **do**
3: Learn a base classifier h_t based on $\mathcal{D}$
4: **end for**
5: Step 2: Construct new data sets from $\mathcal{D}$
6: **for** $i \leftarrow 1$ to m **do**
7: Construct a new data set that contains $\{\mathbf{x}'_i, y_i\}$, where $\mathbf{x}'_i = \{h_1(\mathbf{x}_i), h_2(\mathbf{x}_i), \dots, h_T(\mathbf{x}_i)\}$
8: **end for**
9: Step 3: Learn a second-level classifier
10: Learn a new classifier h' based on the newly constructed data set
11: **return** $H(\mathbf{x}) = h'(h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_T(\mathbf{x}))$

Build T base models (with a good bias variance tradeoff) $\{h_t\}_{i=1}^T$

$\forall (x_i, y_i)$ in D
$D' = \{x_i', y_i\}$ where $x_i' = \langle h_1(x_i), \dots, h_T(x_i) \rangle$

meta classifier on D'
$H(x) = h'(h_1(x) + \dots + h_T(x))$

```
from sklearn import model_selection
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from mlxtend.classifier import StackingClassifier
import numpy as np
import warnings

warnings.simplefilter('ignore')

clf1 = KNeighborsClassifier(n_neighbors=1)
clf2 = RandomForestClassifier(random_state=1)
clf3 = GaussianNB()
lr = LogisticRegression()
sclf = StackingClassifier(classifiers=[clf1, clf2, clf3],
                          meta_classifier=lr)

print('3-fold cross validation:\n')

for clf, label in zip([clf1, clf2, clf3, sclf],
                      ['KNN',
                       'Random Forest',
                       'Naive Bayes',
                       'StackingClassifier']):

    scores = model_selection.cross_val_score(clf, X, y,
                                             cv=3, scoring='accuracy')
    print("Accuracy: %0.2f (+/- %0.2f) [%s]"
          % (scores.mean(), scores.std(), label))
```

Key points:
- The more different the base classifiers are, the better stacking tends to work.
- Stacking works really good in Kaggle competitions.
- In the real world, stacking is seldom used as compared to boosting and bagging.
  The training and run-time is too much.